



National Forensic Sciences University, Gandhinagar

School of Cyber Security and Digital Forensics

Assignment:

Practical - 9 : Snort Rule Creation

Name: Aastha Thakker

Roll No. 250103002004

Sem: 2

Subject: Network Security

Course: MSc Cyber Security

Batch: 2025 -2027

Lab Overview & Objectives

Aim: To deploy a Network Intrusion Detection System (NIDS) using Snort to detect common reconnaissance and scanning activities such as ICMP ping sweeps and Nmap scans.

Objectives

- Install and configure Snort as a NIDS sensor on Ubuntu.
- Configure Snort's network variables and rule paths.
- Write custom Snort rules to detect ICMP ping floods, TCP connect scans, SYN scans, and port scan activity.
- Enable and use built-in community rules (scan.rules).
- Validate the configuration and run Snort in alert console mode.
- Generate test traffic from Kali Linux and verify alerts are triggered on the NIDS sensor.

Required Tools & Environment

Component	IP Address	Role	Key Configuration
Kali Linux	192.168.101.129	Attacker	Initiates scans and attacks
Metasploitable 2	192.168.101.130	Target	Vulnerable victim server
Ubuntu Server	192.168.101.128	NIDS Sensor	Snort installed; Promiscuous Mode enabled

All three VMs must be on the same Internal Network segment in VirtualBox (e.g., 'SnortLab').

Promiscuous Mode must be enabled on the Ubuntu VM's network adapter to capture traffic meant for other machines.

Theory

Every Snort rule is a single line of text composed of two main parts: the Rule Header and the Rule Options. Understanding this structure is fundamental to writing effective detection rules.

The Rule Header (What, Where & When)

The Rule Header is the first part of the rule and defines:

- Action: What Snort should do (e.g., alert, log, drop, reject, pass).
- Protocol: The traffic protocol to inspect (tcp, udp, icmp, ip).
- Source IP / Port: Where the traffic originates.
- Direction Operator: The arrow (-> or <>) indicating traffic direction.
- Destination IP / Port: Where the traffic is headed.

```
<action> <protocol> <src_IP> <src_port> -> <dst_IP> <dst_port>
```

Component	Description	Example
Rule Action	The action Snort performs when the rule is triggered.	alert
Protocol	The network protocol the rule applies to.	tcp, udp, icmp, ip
Source IP	The IP address of the sender.	\$EXTERNAL_NET, 192.168.1.10, any
Source Port	The port number of the sender.	any, 1024: (ports > 1024)
Direction Operator	Defines the flow of traffic.	-> (Unidirectional) or <> (Bidirectional)
Destination IP	The IP address of the receiver.	\$HOME_NET, 10.0.2.5, any
Destination Port	The port number of the receiver.	80, 22, any

Rule Action

Action	Description
alert	Generates an alert and logs the packet. (Most common for NIDS).
log	Logs the packet without generating an alert.
pass	Ignores the packet, stopping inspection on it. (Used in IPS mode to allow known traffic).
drop	Blocks (drops) the packet and logs it. (Used in IPS mode).

The Rule Options (Why & How)

The Rule Options are enclosed in parentheses () and provide detection specifics. Multiple options are separated by semicolons (;). Common options include:

Option	Purpose
<code>msg</code>	Human-readable alert message shown in output.
<code>sid</code>	Unique Snort rule identifier (local rules use 1000000+).
<code>rev</code>	Rule revision number for tracking updates.
<code>flags</code>	Matches specific TCP flags (e.g., S = SYN, A = ACK).
<code>itype</code>	Matches specific ICMP type (e.g., 8 = Echo Request).
<code>flow</code>	Controls state tracking (e.g., stateless for SYN scans).
<code>detection_filter</code>	Rate-based triggering — only alert after N events in T seconds.
<code>threshold</code>	Limits alert frequency to avoid flooding the log.

Environmental Setup

VirtualBox Network Configuration

For Snort to intercept traffic between the attacker and the target (Kali → Metasploitable), all three VMs must share the same network segment.

1. In VirtualBox, go to each VM's Settings → Network.
2. Set Attached to: Internal Network.
3. Name the network (e.g., SnortLab) — all three VMs must use the same name.
4. For the Ubuntu VM, expand Advanced and set Promiscuous Mode to Allow All.
5. Boot all three VMs and verify IP addresses.

System Preparation on Ubuntu (NIDS Sensor)

Update the system and install networking tools:

```
> sudo apt update && sudo apt upgrade -y
```

```
> sudo apt install net-tools -y
```

Identify your primary network interface name:

```
> ip a
```

Enabling Promiscuous Mode (OS Level)

This command allows the network card to capture all traffic on the segment, not just packets addressed to the Ubuntu machine's own MAC address. Replace `enp0s1` with your actual interface name in all subsequent commands.

```
sudo ip link set enp0s1 promisc on
```

Verify Promiscuous Mode is active, look for the PROMISC flag in the output of `'ip a'`

```
(addict@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:c8:3d:83 brd ff:ff:ff:ff:ff:ff
    inet 192.168.101.129/24 brd 192.168.101.255 scope global dynamic noprefixroute eth0
        valid_lft 1629sec preferred_lft 1629sec
    inet6 fe80::20c:29ff:fec8:3d83/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

Kali's IP

```

msfadmin@metasploitable:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 16436 qdisc noqueue
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        inet6 ::1/128 scope host
            valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast qlen 1000
    link/ether 00:0c:29:36:68:e2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.101.130/24 brd 192.168.101.255 scope global eth0
        inet6 fe80::20c:29ff:fe36:68e2/64 scope link
            valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop qlen 1000
    link/ether 00:0c:29:36:68:ec brd ff:ff:ff:ff:ff:ff

```

Metasploitable IP

```

ubuntu@ubuntu:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,PROMISC,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:9b:4a:84 brd ff:ff:ff:ff:ff:ff
    inet 192.168.101.128/24 brd 192.168.101.255 scope global dynamic noprefixroute ens33
        valid_lft 1592sec preferred_lft 1592sec
    inet6 fe80::20c:29ff:fe9b:4a84/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

Ubuntu's IP

Snort Installation & Configuration

Installing Snort

Install Snort using the APT package manager: `sudo apt install snort -y`

If Snort is already installed, verify the version: `snort -v`

Editing the Snort Configuration File

The main Snort configuration file is located at `/etc/snort/snort.conf`. We need to update two key network variables.

Step A: Set HOME_NET

Open the configuration file: `sudo nano /etc/snort/snort.conf`

Find the line: `ipvar HOME_NET any`

Change it to your actual network subnet (example): `ipvar HOME_NET 192.168.101.0/24`

Step B : Set EXTERNAL_NET

Find this line: `ipvar EXTERNAL_NET any`

Change it to: `ipvar EXTERNAL_NET !$HOME_NET`

This tells Snort that everything outside the HOME_NET is considered external/untrusted traffic.

Enabling Rule Sets

Scroll to the bottom of `snort.conf` where rule files are included. Ensure the following lines are uncommented (remove the leading `#` if present):

`include $RULE_PATH/local.rules`

`include $RULE_PATH/scan.rules`

Rule File	Description
<code>local.rules</code>	User-defined custom rules
<code>scan.rules</code>	Built-in community rules for detecting port scans and reconnaissance.

This was all done before in another lab.

Writing Custom Snort Rules

Open the local rules file: `sudo nano /etc/snort/rules/local.rules`. Add the following four rules one by one.

Rule 1: Detect ICMP Ping Flood

```
> alert icmp any any -> any any (msg: "ICMP Ping Flood Detected"; sid:1000001; rev:1;)
```

Rule 2: Detect TCP Connect Scan (SSH Port 22)

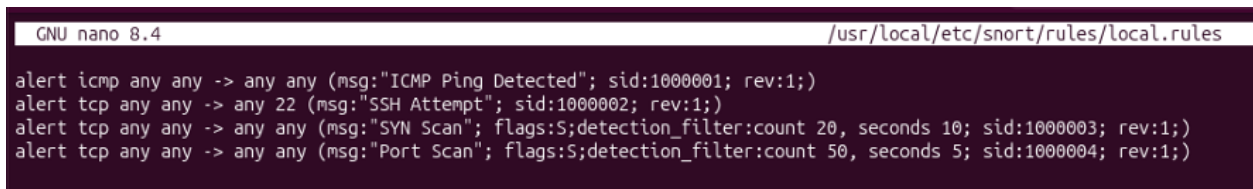
```
> alert tcp any any -> any 22 (msg: "SSH Attempt"; sid:1000002; rev:1;)
```

Rule 3: Detect Nmap SYN Scan

```
> alert tcp any any -> any any (msg: "SYN Scan"; flags:S:detection_filter:count 20, seconds 10; sid:1000003; rev:1;)
```

Rule 4: Detect Aggressive Port Scan

```
> alert tcp any any -> any any (msg: "Port Scan"; flags:S:detection_filter:count 50, seconds 5; sid:1000004; rev:1;)
```



```
GNU nano 8.4 /usr/local/etc/snort/rules/local.rules
alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
alert tcp any any -> any 22 (msg:"SSH Attempt"; sid:1000002; rev:1;)
alert tcp any any -> any any (msg:"SYN Scan"; flags:S:detection_filter:count 20, seconds 10; sid:1000003; rev:1;)
alert tcp any any -> any any (msg:"Port Scan"; flags:S:detection_filter:count 50, seconds 5; sid:1000004; rev:1;)
```

Snapshot of all rules

Execution and Alert Verification

Validate Snort Configuration

This critical step checks all syntax in `snort.conf` and all included rule files. Snort will not start if validation fails.

```
> sudo snort -T -c /etc/snort/snort.conf -i enp0s1
```

Expected Output: Snort successfully validated the configuration! If you see errors, they will indicate the file and line number. Correct them before proceeding.

Start Snort in Alert Console Mode

On the Ubuntu NIDS sensor, launch Snort with the following options:

```
> sudo snort -A console -q -c /etc/snort/snort.conf -i enp0s1
```

Flag	Meaning
<code>-A console</code>	Print alerts to the terminal (real-time output).
<code>-q</code>	Quiet mode — suppresses startup banner; shows only alerts.
<code>-c ...</code>	Specifies the path to the Snort configuration file.
<code>-i enp0s1</code>	Specifies the network interface to listen on.

Snort will start silently and wait for incoming traffic. Keep this window open.

A) Test 1: ICMP Ping Flood

1. A ping (ICMP echo request) checks whether a system is online. Attackers often use it during reconnaissance to identify active machines before launching further attacks.
1. This rule tells Snort to alert on any ICMP traffic from any source to any destination. When a ping packet is detected, Snort matches the ICMP protocol and generates the “ICMP Ping Detected” alert.
2. Type

```
> ping <ip> or
> hping3 -icmp -flood <ip>
```

```

(addict@kali) ~$ ping 192.168.101.130
PING 192.168.101.130 (192.168.101.130) 56(84) bytes of data.
64 bytes from 192.168.101.130: icmp_seq=1 ttl=64 time=0.883 ms
64 bytes from 192.168.101.130: icmp_seq=2 ttl=64 time=1.11 ms
64 bytes from 192.168.101.130: icmp_seq=3 ttl=64 time=1.04 ms
64 bytes from 192.168.101.130: icmp_seq=4 ttl=64 time=0.854 ms
64 bytes from 192.168.101.130: icmp_seq=5 ttl=64 time=1.09 ms
64 bytes from 192.168.101.130: icmp_seq=6 ttl=64 time=0.998 ms
64 bytes from 192.168.101.130: icmp_seq=7 ttl=64 time=1.04 ms
64 bytes from 192.168.101.130: icmp_seq=8 ttl=64 time=1.15 ms
64 bytes from 192.168.101.130: icmp_seq=9 ttl=64 time=1.01 ms
64 bytes from 192.168.101.130: icmp_seq=10 ttl=64 time=0.945 ms

```

```

04/03-19:11:13.921570 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:13.921575 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:14.942878 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:14.943054 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:15.943975 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:15.944571 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:16.945278 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:16.945619 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:17.945784 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:17.946079 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:18.947122 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:18.947140 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:19.948191 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:19.948764 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129
04/03-19:11:20.950186 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.129 -> 192.168.101.130
04/03-19:11:20.950196 [**] [1:1000001:1] "ICMP Ping Detected" [**] [Priority: 0] {ICMP} 192.168.101.130 -> 192.168.101.129

```

B) Test 2 : TCP Connect Scan (SSH Port 22)

1. Detects attempts to connect to SSH service (port 22) Could indicate:
 - Legitimate login
 - Brute-force attack
 - Unauthorized access attempt
2. How Snort Detected It. Matches:
 - TCP protocol
 - Destination port 22
3. Any connection to port 22 triggers alert
4. This Nmap TCP connect scan attempts a full TCP handshake to port 22. The SYN flag on the initial packet matches Rule 2 and triggers the alert.

```

(addict@kali) ~$ nmap -p 22 192.168.101.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-03 19:12 +0530
Nmap scan report for 192.168.101.130
Host is up (0.00047s latency).

PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:36:68:E2 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds

```

```

04/03-19:11:37.630599 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:39397 -> 192.168.101.130:22
04/03-19:11:44.630526 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:57823 -> 192.168.101.130:22
04/03-19:11:44.631119 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:57823 -> 192.168.101.130:22
04/03-19:11:46.494689 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:50730 -> 192.168.101.130:22
04/03-19:11:46.495378 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:50730 -> 192.168.101.130:22
04/03-19:11:48.427167 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:37579 -> 192.168.101.130:22
04/03-19:11:48.427943 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:37579 -> 192.168.101.130:22
04/03-19:11:55.786821 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:47821 -> 192.168.101.130:22
04/03-19:11:55.788535 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:47821 -> 192.168.101.130:22
04/03-19:12:17.489064 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:48388 -> 192.168.101.130:22
04/03-19:12:17.489927 [**] [1:1000002:1] "SSH Attempt" [**] [Priority: 0] {TCP} 192.168.101.129:48388 -> 192.168.101.130:22

```

C) Test 3 : Nmap SYN Scan

1. A SYN scan sends TCP packets with only the SYN flag set to discover open ports. It does not complete the full TCP handshake, making it a common scanning technique used by tools like Nmap.
2. The rule monitors TCP traffic and specifically looks for packets with the SYN flag set (flags:S). When such a packet is seen, Snort triggers the "SYN Scan" alert.
3. Type: *nmap -sS -F <ip>*. The -sS flag performs a stealth SYN (half-open) scan, and -F limits it to the 100 most common ports for speed. The flood of SYN packets triggers Rule 3. The built-in scan.rules may also generate: SCAN nmap TCP scan detected.

```

$ nmap -sS -F 192.168.101.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-03 19:13 +0530
Nmap scan report for 192.168.101.130
Host is up (0.0011s latency).
Not shown: 82 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
513/tcp   open  login
514/tcp   open  shell
2049/tcp  open  nfs
2121/tcp  open  ccproxy-ftp
3306/tcp  open  mysql
5432/tcp  open  postgresql
5900/tcp  open  vnc
6000/tcp  open  X11
8009/tcp  open  ajp13
MAC Address: 00:0C:29:36:68:E2 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.72 seconds

```

```

04/03-19:13:31.543531 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:1027
04/03-19:13:31.543532 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:49156
04/03-19:13:31.543533 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:1026
04/03-19:13:31.543533 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:8443
04/03-19:13:31.543612 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:8081
04/03-19:13:31.543844 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:8009
04/03-19:13:31.544082 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:4899
04/03-19:13:31.544230 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:631
04/03-19:13:31.544305 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:2717
04/03-19:13:31.544318 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:1029
04/03-19:13:31.544399 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:5800
04/03-19:13:31.544400 [**] [1:1000003:1] "SYN Scan" [**] [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:9999

```

D) Test 4 : Aggressive Port Scan

1. An aggressive scan across all 65535 ports (or at high packet rate) easily exceeds the 50 SYN packets per 5 seconds threshold, triggering Rule 4.
2. A port scan checks multiple ports quickly to find:
 - Open services
 - Vulnerabilities
3. How Snort Detected It
 - flags:S → SYN packets (start of connections)
 - detection_filter:count 50, seconds 5 → 50 SYN packets within 5 seconds
 - High-speed scanning behavior triggers alert
4. Type `nmap -sS -p- <ip>` OR `nmap -sS --min-rate 1000 <ip>`

```
$ nmap -p 22 192.168.101.130
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-03 19:12 +0530
Nmap scan report for 192.168.101.130
Host is up (0.00047s latency).
```

```
PORT      STATE SERVICE
22/tcp    open  ssh
MAC Address: 00:0C:29:36:68:E2 (VMware)
```

```
Nmap done: 1 IP address (1 host up) scanned in 0.70 seconds
```

```
04/03-19:13:31.546777 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:444
04/03-19:13:31.546785 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:9
04/03-19:13:31.546785 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:9
04/03-19:13:31.546785 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:49154
04/03-19:13:31.546785 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:49154
04/03-19:13:31.546786 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:514
04/03-19:13:31.546786 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:514
04/03-19:13:31.546786 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:5060
04/03-19:13:31.546786 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:5060
04/03-19:13:31.546867 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:5190
04/03-19:13:31.546867 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:5190
04/03-19:13:31.546867 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:543
04/03-19:13:31.546867 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:543
04/03-19:13:31.546868 ** [1:1000004:1] "Port Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:6646
04/03-19:13:31.546868 ** [1:1000003:1] "SYN Scan" ** [Priority: 0] {TCP} 192.168.101.129:58963 -> 192.168.101.130:6646
```

Troubleshoots

Problem	Solution
Alerts only appear when scanning Ubuntu's IP, not Metasploitable.	VirtualBox Promiscuous Mode is not set to 'Allow All'. Check VM Settings → Network → Advanced → Promiscuous Mode.
snort -T validation fails with errors.	Check the exact error line. Usually a typo in snort.conf or a missing/malformed rule in local.rules. Verify semicolons and parentheses.
No alerts appear at all.	Confirm the correct interface name with 'ip a'. Ensure rules are uncommented in snort.conf. Verify HOME_NET matches the lab subnet.
hping3 command not found.	Install with: sudo apt install hping3 -y (on Kali).
PROMISC flag missing after running ip link set promisc on.	Run the command with sudo. Verify interface name is exact (case-sensitive).

Conclusion

- Deployed Snort as a Network Intrusion Detection System on Ubuntu in a three-VM virtual lab environment.
- Configured Snort's HOME_NET and EXTERNAL_NET variables to define the network boundary.
- Enabled built-in community rules (scan.rules) for detecting known reconnaissance patterns.
- Authored four custom rules in local.rules to detect ICMP floods, SSH connection attempts, SYN scans, and aggressive port scanning.
- Validated the configuration and confirmed promiscuous packet capture via sniffer mode.
- Generated real attack traffic from Kali Linux and verified that all expected alerts were triggered on the NIDS sensor.
- A NIDS is only as effective as its ruleset. Custom rules must be tuned to the specific environment to reduce false positives while ensuring critical threats are always detected.
- Promiscuous Mode (or a network TAP/SPAN port) is essential for a passive NIDS to see traffic not addressed to itself.